

LABORATORY OF EMBEDDED SYSTEMS ARCHITECTURE

Exercise 1

Topic: Introduction to programming Intel Galileo.

1. Laboratory stand

The laboratory stand consists of a PC, Intel evaluation board Galileo Gen2, a set of wiring and basic sensors cooperating with the Base Shield from the Grove Starter Kit Plus Intel IoT Edition (Fig. 1).



Fig.1. A set of sensors and wiring includes in Grove Starter Kit Plus Intel IoT Edition

Grove Starter Kit Plus for IoT Intel Edition includes also:

- transition plate (Base Shield) to enable connection sensors to the Intel Galileo Board,
- alphanumeric LCD display (2 lines of 16 characters) with backlight RGB,
- push button,
- rotary potentiometer,

- electromagnetic relay module,
- buzzer module,
- microphone module,
- light sensor,
- temperature sensor,
- touch sensor,
- servo,
- three LED diodes (blue, green and red) cooperating with LED module Socket Kit,
- battery adapter,
- information brochure,
- 8GB SD card.

2. Programing in Arduino IDE

Turn on PC computer indicated by the teacher. At the time of its boot you can refer to the construction of the Galileo Gen 2 Intel board. Locate power supply socket and the USB Client port. Gently slide the Grove Base Shields into the slot on Intel Galileo Gen Board.

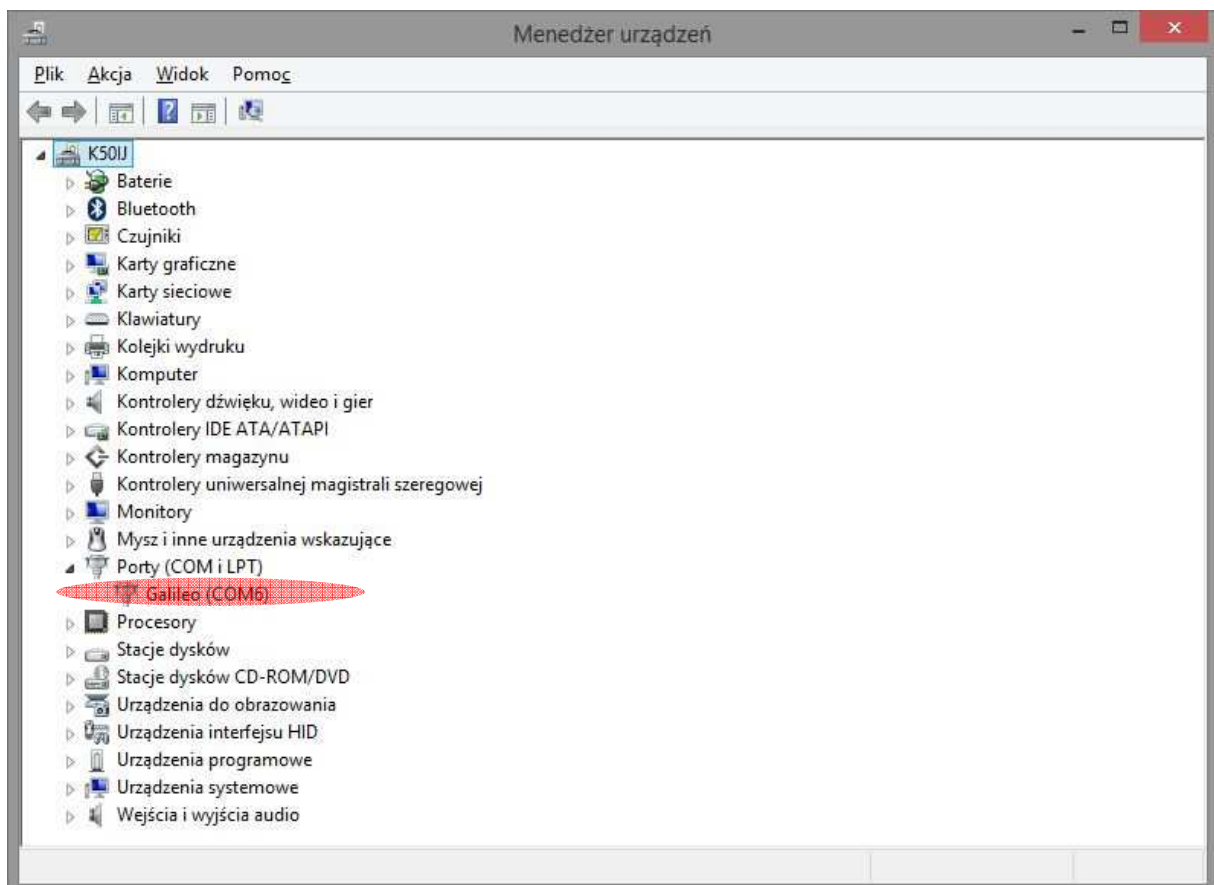


Fig. 2. Device Manager in Windows 8.1.

Connect the power supply to the Intel Galileo Gen2 board, and after that connect the USB cable to the computer using the USB client port (micro USB).

Note 1!

Keep the connection sequence of the power supply and USB cable. Reversing of the order wiring connection can damage Intel Galileo Board.

In the Device Manager in the *Porty* field (COM & LPT) check the COM port number at which the Intel Galileo Gen2 is visible. (Fig. 2).

Find a shortcut to the Arduino IDE, which should be on the Windows desktop and start the program. On the *Narzędzia* menu, scroll to the Intel platform Galileo Gen2 as cooperating with the Arduino IDE environment as shown in Figure 3.

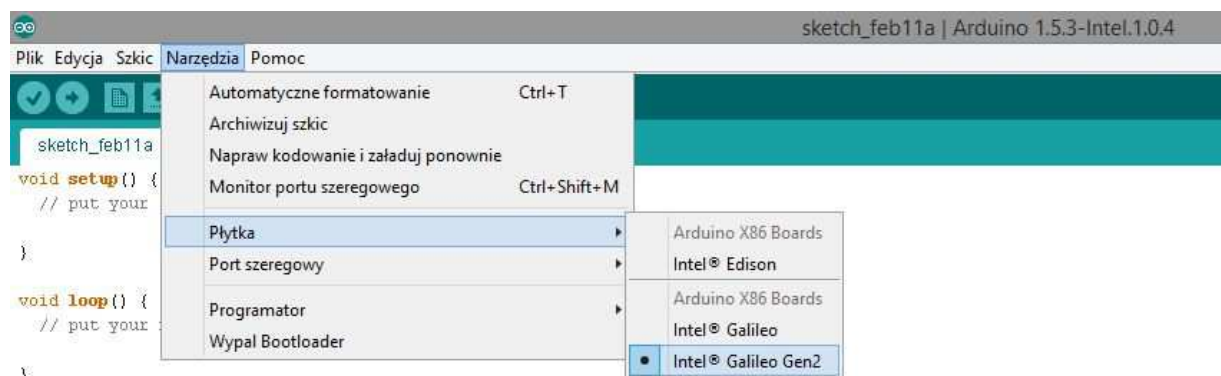


Fig. 3. Structure of menu *Narzędzia* of the Arduino IDE

Then, using the menu *Narzędzia* -> *Port szeregowy*, select the right COM port, to which the motherboard of Intel Galileo Gen2 - read earlier in the Windows Device Manager -is connected.

From the menu *Plik* -> *Przykłady* -> *01.Basics* (Fig. 4) load the Blink program. This is the simplest example of a program whose goal is blinking LED diode, located on the Galileo Intel Gen2 board on the left side from the USB HOST port. This LED diode is marked with symbol "L".

Compiling the programs (the so-called Sketch-s) in the Arduino IDE and their upload to the target board follow after pressing the left arrow icon (Fig. 4).

Examine the code and run the sample program (sketch) Blink and observe its operation. Modify the program by changing ON and OFF time of the LED diode "L" and changing sequences of flashing.

Intel Galileo Gen2 has two ports of UART type. The first (UART0) is derived for 2-pin connector pin (female) compatible with the Arduino platform (pin 0-RX, Pin 1-TX). The second port (UART1) is derived for 6-pin connector pin (male – goldpin type) located next to the Ethernet port. It is used mainly for communication with Linux Yocto, but it can also be used in the Arduino IDE. To use of this interface, it is necessary to use the USB compatible FTDI (or equivalent), wherein the signal converter UART TTL 3.3V for USB is integrated (Fig. 5).

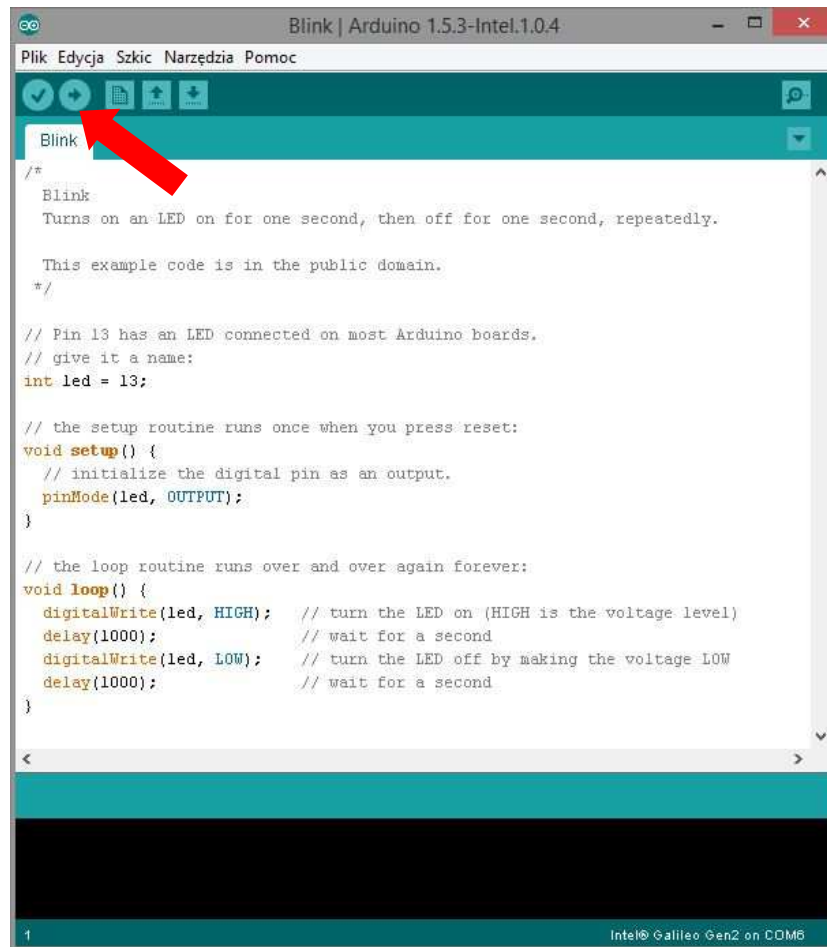


Fig.4. Sample program (sketch) in Arduino IDE



Fig. 5. The way to connect FTDI cable (UART TTL 3,3V->USB) to Intel Galileo Gen2 platform

Connect the FTDI cable as shown in Figure 5, making sure that the black wire is connected with terminal GND pin connector. Plug the USB cable to the PC port. Check in the Device Manager of Windows COM port number from which UART1 interface of Intel Galileo Gen2 platform will be seen.

Start the terminal program Tera Term, and then from menu *File* select *New connection* option. In the window that appears, select *Serial* and select the COM port previously read in the Device Manager (Fig.6). Then, from the *Setup* menu, select *Serial port* and set the baud rate to 115200 baud as shown in Figure 7. In this way, Tera Term terminal program is configured to work with Intel Galileo Gen2.

Note 2!

In sketches written in the Arduino IDE serial port UART0 it is mapped to a Serial port, and serial port UART1 to a SERIAL2 port. Port serial1 is not available!

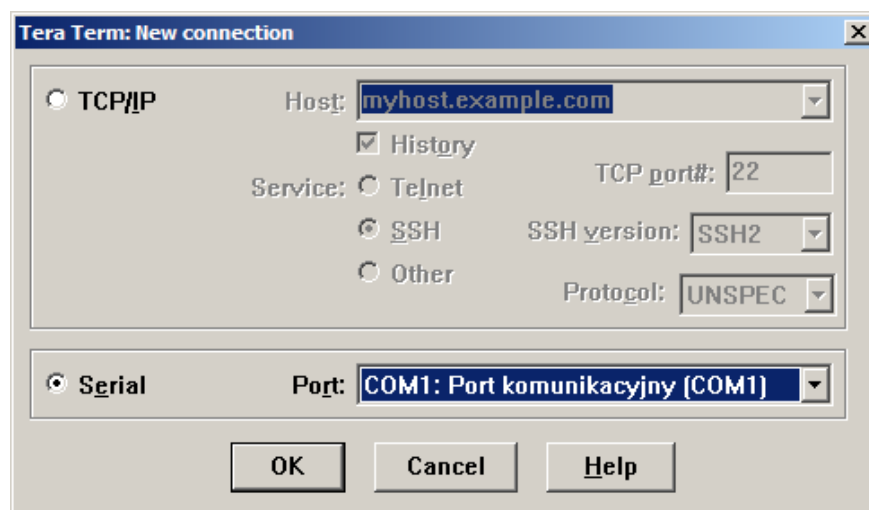


Fig.6. Menu New connection Tera Term program

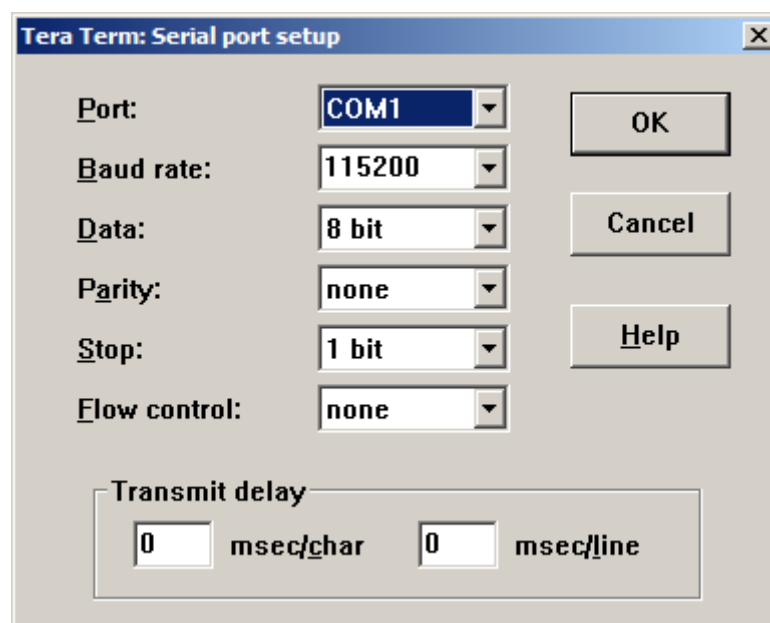


Fig. 7. Menu Serial port setup Tera Term program

In the next part of the lab analyze sample programs provided by the producer of Grove Starter Kit Plus Intel IoT Edition. These programs can be found in the menu *Plik -> Przykłady -> GroveStarterKitV2*. When analyzing individual programs use the elements of the starter kit, which connect according to the booklet that came with the kit, using the supplied cables. Modify in accordance with Note 2 examples of programs that use UART interface (Serial Arduino IDE) and also make sure that the baud rate in programs Arduino IDE and in the terminal program Tera Term is the same.

Note 3!

Before connecting more elements to the transition board (Base shield) it is recommended to turn the power off of Intel Galileo board, and before unplugging the USB cable CLIENT (micro USB Connector).

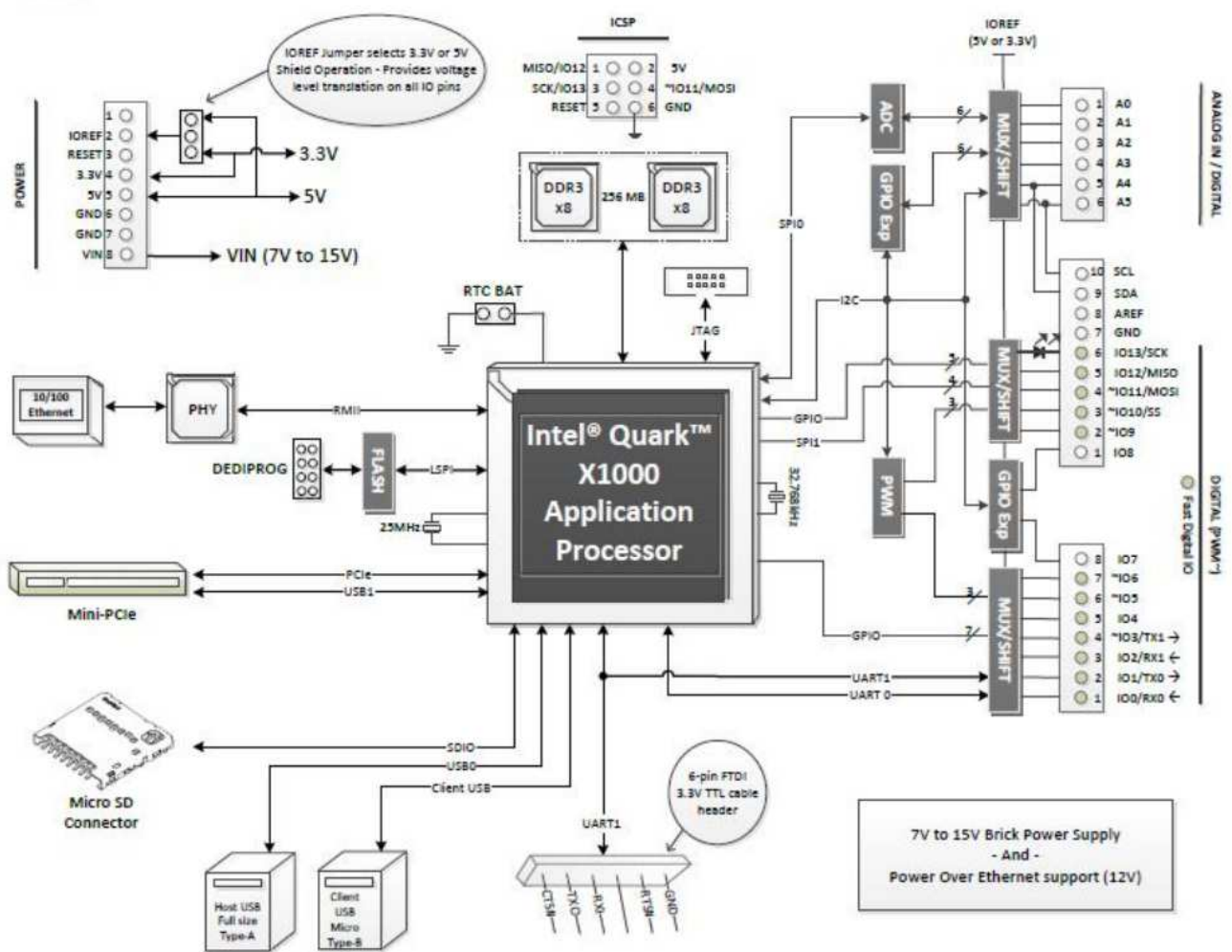


Fig.8. The block diagram of the Intel Galileo Gen2.

In the protocol note the functions syntaxes, which will be useful later in laboratory:

- read the status of the buttons,
- set the status on the digital ports,
- read values from analog-to-digital converter,

- d) configuration of PWM channels,
- e) displaying text on the LCD display and setting backlight RGB color.

3. Programming in Bash (Linux Yocto)

Controlling Galileo Intel platform states of ports in Linux is possible through the use of virtual file system sysfs. It should be noted, that due to the difference in the construction of the Galileo Gen1 platform (using chip Cypress CY8C9540A) and Galileo Gen2 (using chip PCAL9555A) programming both platforms at the level of Linux is slightly in different way. This study is limited to discussion of second generation Galileo platform (Fig 8).

Connect the FTDI as shown in Figure 5 and run Tera Term program, and then configure it to connection via the serial port with a speed of 115,200 bps (Fig. 6 and 7). Connect the power of the Intel Galileo Gen2 board. Wait to load a Linux Yocto located in Flash memory. Log in to the operating system as: root.

GPIO ports status of the Galileo Intel platform running under Linux can be checked in the file `"/sys/kernel/debug/gpio"`. To read its contents issue the command in terminal (in the program window Tera Term):

```
less /sys/kernel/debug/gpio
```

```

GPIOs 0-1, platform/sch_gpio.2398, sch_gpio_core:
GPIOs 2-7, platform/sch_gpio.2398, sch_gpio_resume:
GPIOs 8-15, intel_qrk_gip_gpio:
  gpio-8  (SPI_CS      ) out hi
  gpio-9  (pca19555a-exp2-int ) in  hi
GPIOs 16-31, i2c/0-0025, pca19555a, can sleep:
GPIOs 32-47, i2c/0-0026, pca19555a, can sleep:
  gpio-47 (sysfs      ) out hi
GPIOs 48-63, i2c/0-0027, pca19555a, can sleep:
  gpio-63 (sysfs      ) in  hi
GPIOs 64-79, pca9685-gpio, can sleep:
~
~
~
~
/sys/kernel/debug/gpio

```

Fig. 9. Contents of the file GPIO

The effect of the command will be listing of the file's contents as shown in Figure 9. It should be noted that the Intel Galileo Gen2 platform has 79 ports, but not all are routed to connectors compatible with Arduino. Most of them are used to configure the ports compatible with Arduino as shown in Table 1. Some ports are native and controlled directly from Intel Quark, others are

connected by intermediate device such as PCAL9555 and PCA9685 (block GPIOexp, PWM, MUX, SHIFT on figure 9).

Table 1. Configuration of Arduino-compatible ports on Galileo Gen2 platform

Shield pin	Function	Linux	Level Shifter GPIO L: dir_out H: dir_in I: *	22k Pull-Up GPIO L: pulldown H: pullup I: off	Pin Mux 1 GPIO	Pin Mux 2 GPIO	Interrupt modes L: low-level H: high-level R: rising-edge F: falling-edge B: both edges
I00	UART0 RX	ttyS0	gpio32	gpio33	-	-	-
	GPIO	gpio11			-	-	L/H/R/F
I01	UART0 TX	ttyS0	gpio28	gpio29	gpio45 (H)	-	-
	GPIO	gpio12			gpio45 (L)	-	L/H/R/F
I02	UART1 RX	ttyS1	gpio34	gpio35	gpio77 (H)	-	-
	GPIO	gpio13			gpio77 (L)	-	L/H/R/F
	GPIO	gpio61	-		gpio77 (L)	-	R/F/B
I03	UART1 TX	ttyS1	gpio16	gpio17	gpio76(H)	-	-
	GPIO	gpio14			gpio76(L)	gpio64(L)	L/H/R/F
	PWM	pwm1			gpio76(L)	gpio64(H)	-
	GPIO	gpio62	-		gpio76(L)	gpio64(L)	R/F/B
I04	GPIO	gpio6	gpio36	gpio37	-	-	R/F/B
I05	GPIO	gpio0	gpio18	gpio19	gpio66(L)	-	R/F/B
	PWM	pwm3			gpio66(H)	-	-
I06	GPIO	gpio1	gpio20	gpio21	gpio68(L)	-	R/F/B
	PWM	pwm5			gpio68(H)	-	-
I07	GPIO	gpio38	-	gpio39	-	-	-
I08	GPIO	gpio40	-	gpio41	-	-	-
I09	GPIO	gpio4	gpio22	gpio23	gpio70(L)	-	R/F/B
	PWM	pwm7			gpio70(L)	-	-
I010	GPIO	gpio10	gpio26	gpio27	gpio74(L)	-	L/H/R/F
	PWM	pwm11			gpio74(H)	-	-
I011	GPIO	gpio5	gpio24	gpio25	gpio44(L)	gpio72(L)	R/F/B
	SPI MOSI	spidev1.0			gpio44(H)	gpio72(L)	-
	PWM	pwm9			-	gpio72(H)	-
I012	GPIO	gpio15	gpio42	gpio43	-	-	L/H/R/F
	SPI MISO	spidev1.0			-	-	-
I013	GPIO	gpio7	gpio30	gpio31	gpio46(L)	-	R/F/B
	SPI SCK	spidev1.0			gpio46(H)	-	-
I014	GPIO	gpio48	-	gpio49	-	-	R/F/B
	ADC A0	in_voltage0_raw			-	-	-
I015	GPIO	gpio50	-	gpio51	-	-	R/F/B
	ADC A1	in_voltage1_raw			-	-	-
I016	GPIO	gpio52	-	gpio53	-	-	R/F/B
	ADC A2	in_voltage2_raw			-	-	-
I017	GPIO	gpio54	-	gpio55	-	-	R/F/B
	ADC A3	in_voltage3_raw			-	-	-
I018	GPIO	gpio56	-	gpio57	gpio60(H)	gpio78(H)	R/F/B
	ADC A4	in_voltage4_raw			gpio60(H)	gpio78(L)	-
	I2C SDA	i2c-0			gpio60(L)	-	-
I019	GPIO	gpio58	-	gpio59	gpio60(H)	gpio79(H)	R/F/B
	ADC A5	in_voltage5_raw			gpio60(H)	gpio79(L)	-
	I2C SCL	i2c-0			gpio60(L)	-	-

There are the following designations in the table:

- "L" – GPIO port is configured as output in low state
- "H" – GPIO port is configured as output in high state
- "I" – GPIO port is configured as input in high impedance state

Write digital ports

The sample configuration IO13 port that is connected to the LED "L" on the Galileo Gen2 board is as follows:

According to Table 1 IO13 port is connected to the gpio7 Intel Quark port, so we export gpio7 port to be able to be controlled with a virtual file system sysfs:


```
echo -n "7" > /sys/class/gpio/export
```

Next we set gpio7 port as output port:

```
echo -n "out" > /sys/class/gpio/gpio7/direction
```

Sequentially we set high state on gpio7 port to turn on the LED diode "L":

```
echo -n "1" > /sys/class/gpio/gpio7/value
```

Is LED diode "L" turn on? If not, it is still necessary to configure multiplexers (Fig 8), which mediate between the microprocessor ports and connectors compatible with Arduino. To do this, according to Table 1, the processor gpio30 ports and gpio46 should be configured as output in the low state. While using the IO13 port as output configuration resistors of pullup and pulldown types (using gpio31) can be omitted. Therefore, issue the commands:

```
echo -n "30" > /sys/class/gpio/export  
echo -n "46" > /sys/class/gpio/export  
echo -n "out" > /sys/class/gpio/gpio30/direction  
echo -n "out" > /sys/class/gpio/gpio46/direction  
echo -n "0" > /sys/class/gpio/gpio30/value  
echo -n "0" > /sys/class/gpio/gpio46/value
```

The above command should result in the illumination of the LED "L" placed next to the USB HOST port and connected to the port IO13 compatible with Arduino. The state of this port can now be changed by changing only the status on Intel Quark gpio7 port.

```
echo -n "0" > /sys/class/gpio/gpio7/value  
echo -n "1" > /sys/class/gpio/gpio7/value
```

With the command:

```
less /sys/kernel/debug/gpio
```

check states of gpio ports after configuration of IO13 port after above commands are executed, compare them with Figure 9, the differences write in protocol performed exercise.

Read digital ports

Each port IO compatible with Arduino can be programmed as input. For example for IO2 port issue the commands:

```
echo -n "13" > /sys/class/gpio/export  
echo -n "34" > /sys/class/gpio/export  
echo -n "35" > /sys/class/gpio/export  
echo -n "77" > /sys/class/gpio/export  
echo -n "in" > /sys/class/gpio/gpio13/direction  
echo -n "out" > /sys/class/gpio/gpio34/direction  
echo -n "out" > /sys/class/gpio/gpio35/direction  
echo -n "out" > /sys/class/gpio/gpio77/direction
```

```
echo -n "1" > /sys/class/gpio/gpio34/value  
echo -n "0" > /sys/class/gpio/gpio35/value  
echo -n "0" > /sys/class/gpio/gpio77/value
```

State of IO2 port can be read with command:

```
cat /sys/class/gpio/gpio13/value
```

In order to better illustrate the changes in the IO2 port connect to the Base Shield from the Grove starter kit Button switch or Touch to the D2 port and issue the following command in Terminal (in the Tera Term program).

```
while [ 1 ]  
do  
cat /sys/class/gpio/gpio13/value  
done
```

By changing the button state observe the values returned by Linux. In the protocol answer the questions. Does the switch connect port IO2 to ground or to a power line?? In such a situation do we use pullup or pulldown resistors?

Note 4!

Loops can be interrupted by pressing Ctrl + C.

Reading the states of analog inputs (ADCs)

Analog inputs of Intel Galileo platform utilize analog-to-digital converter AD7298. This is an 8-channel transmitter, but only 6 channels connected to the ports A0-A5 Arduino-compatible are used. Used ADC has a resolution of 12 bits, therefore returns a value in the range of 0-4095.

From Linux reading an analog input (eg. A0) is implemented, as well as digital inputs, using the sysfs virtual file system by using the command:

```
cat /sys/bus/iio/devices/iio\:device0/in_voltage0_raw
```

However, earlier properly configured connections of analog port compatible with Arduino standard with the input analog-to-digital by appropriate multiplexers should be checked. As indicated in Table 1 ports A0-A3 are by default configured as analog inputs. Ports A4 and A5 require the configuration like as in the case of digital inputs and outputs.

Connect the potentiometer from set of Grove Starter Kit Plus Intel IoT Edition to A0 port. Run the following script:

```
while [ 1 ]
do
cat /sys/bus/iio/devices/iio\:device0/in_voltage0_raw
done
```

By changing the position of the potentiometer observe the data returned by the analog-to-digital converter. Save in the protocol extreme values from the transmitter. Disconnect potentiometer and observe and write down what the values will return the ADC converter. For analog port A0 using the Intel Quark port gpio49 connect pullup resistors and next pulldown and observe how they impact on the performance of the transmitter in case the connected and disconnected potentiometer. Make a note in protocol.

Programming of PWM (Pulse-Width Modulation) channel

The PWM signal fulfills a variety of functions in embedded systems. Mostly it is applied to control the brightness of the LEDs, the rotation speed of motors or servomechanism control. Galileo Gen2 board has a 16-channel 12-bit PWM controller (PCA9685) that is connected to the X1000 Quark processor through the I²C interface. From the 16 channels, which belong to the PCA9685 device only 6 channels are used as PWM outputs that are wired to the connector compatible with the Arduino (Fig.8). The others are used as output GPIO ports (General Purpose Input/Output).

Period of PWM signal in Intel Galileo Gen2 platform is configured simultaneously for all channels and can be in the range of 666,666 to 41,666,666 nanoseconds.

In the protocol calculate the maximum and minimum frequency of the PWM waveform, which can generate Galileo Gen2 platform.

Access to the configuration of PWM channels is available via a file system sysfs and exactly via the directory: /sys/class/PWM/pwmchip0/.

Just as for ordinary GPIO ports to gain access to the channel PWM (eg. Channel 1) it should be exported using the command:

```
echo -n "1" > /sys/class/pwm/pwmchip0/export
```

Then running the channel of first PWM module follows by issuing the command:

```
echo -n "1" > /sys/class/pwm/pwmchip0/pwm1/enable
```

The period of the generated waveform (eg. 20 ms) is given in nanoseconds with the command:

```
echo -n "20000000" > /sys/class/pwm/pwmchip0/device/pwm_period
```

Next, enter the duty cycle, specified in nanoseconds calculated according to the formula:

$$duty_cycle[ns] = pwm_period[ns] \cdot \frac{k_w[\%]}{100\%} \quad (1)$$

where: k_w - duty cycle expressed as a percentage.

For example, for the waveform with duty cycle equals 50% and frequency 50 Hz the command setting the duty cycle takes the form:

```
echo -n "10000000" > /sys/class/pwm/pwmchip0/pwm1/duty_cycle
```

Of course, it is still necessary to configure multiplexers (Fig 8), which mediate between the microprocessor ports and connectors compatible with Arduino. Thus, as shown in Table 1. the entire setup procedure of the first channel PWM so as to generate a waveform with a frequency of 50 Hz with the duty cycle of 50% is as follows:

```
echo -n "16" > /sys/class/gpio/export
echo -n "17" > /sys/class/gpio/export
echo -n "76" > /sys/class/gpio/export
echo -n "64" > /sys/class/gpio/export

echo -n "out" > /sys/class/gpio/gpio16/direction
echo -n "in" > /sys/class/gpio/gpio17/direction
echo -n "out" > /sys/class/gpio/gpio76/direction
echo -n "out" > /sys/class/gpio/gpio64/direction

echo -n "0" > /sys/class/gpio/gpio16/value
echo -n "0" > /sys/class/gpio/gpio76/value
echo -n "1" > /sys/class/gpio/gpio64/value

echo -n "1" > /sys/class/pwm/pwmchip0/export
echo -n "1" > /sys/class/pwm/pwmchip0/pwm1/enable
echo -n "20000000" > /sys/class/pwm/pwmchip0/device/pwm_period
echo -n "10000000" > /sys/class/pwm/pwmchip0/pwm1/duty_cycle
```

In order to validate the generated waveform connect blue LED to the IntelGalileo Gen2 platform via the D3 connector of the Base Shield from Grove Starter Kit. By varying the duty ratio in 0÷100% range watch the lighting of the LED.

4. Final task:

1. Write a program in the Arduino IDE environment, which will implement the function of the heating controller with hysteresis ± 0.5 °C. Reading the current temperature will be implemented using a thermistor with a Grove starter kit. Setting the temperature by 1 °C will be implemented using the momentary Button (T+) and Touch sensor (T-), switching on the heater will be done by the relay module. On the LCD screen will be presented set and measured temperature. Dependences on the measured and set temperature should be visualized as the color of the backlight according to the algorithm: If the actual temperature is lower than the set - red (heater ON), in the reverse situation - blue (heater OFF). If both temperatures are the same – green color.

2. Write Bash script using the Vi editor enabling servomechanism control (port IO5) using a potentiometer connected to the IntelGalileo Gen2 platform (port A1). Take into account the fact that the servomechanism accepts a PWM signal 50 Hz and duty cycle in the range of 4÷12%. To help you can take advantage of proposed or other available online courses of Bash and the vi editor (remember to set access rights for your own script with the command: **chmod + x script_name**, run the script: **./nazwa_skryptu**):

VI:

- <http://lewandowicz.eu/instrukcje/vi.php>
- <http://www.cs.put.poznan.pl/dwawrzyniak/vi.pdf>

Bash:

- <http://dief.republika.pl/main.html>
- http://www.uw-team.org/artykul_bash_kurs.html